



Numerical Approximations for Spectral Properties

We now study different numerical techniques to approximate the eigenvalues and eigenvectors of matrices.

There are 2 main families of methods

- ▶ Iterative methods to approximate one or a couple eigenvalues and eigenvectors.
- ▶ Methods to compute ALL eigenvalues and eigenvectors.



Power Method

One can estimate the largest-in-magnitude eigenvalue λ_1 and it's eigenvector $\mathbf{v}^{(1)}$ via the following.

Definition

Let $\mathbf{x}^{(0)}$ be an initial guess at the eigenvector. The *power method*, or power iteration, is defined by the following iteration:

$$\begin{aligned}\mathbf{z}^{(m)} &= \mathbf{x}^{(m)} / \|\mathbf{x}^{(m)}\|_2 \\ \mathbf{x}^{(m+1)} &= A\mathbf{z}^{(m)} \\ \beta_m &= [\mathbf{z}^{(m)}]^T A\mathbf{z}^{(m)}\end{aligned}$$

Then $\mathbf{z}^{(m)} \rightarrow \mathbf{v}^{(1)}$ and $\beta_m \rightarrow \lambda_1$ as $m \rightarrow \infty$.



Power Method

Theorem

Consider a diagonalizable matrix A with largest-in-magnitude eigenvalue λ_1 and eigenvector $\mathbf{v}^{(1)}$, and assume $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots$. Then the power method converges as

$$\|\mathbf{z}^{(m)} - \mathbf{v}^{(1)}\|_{\infty} \leq c \left| \frac{\lambda_2}{\lambda_1} \right|^m$$

for some constant $c > 0$, provided that the initial condition is not orthogonal to $\mathbf{v}^{(1)}$.

(See notes for proof.)

Power Method

Definition

The *Rayleigh-Ritz Quotient* is defined as

$$\mathcal{R}_A(\mathbf{z}) = \frac{\mathbf{z}^T A \mathbf{z}}{\mathbf{z}^T \mathbf{z}}$$

Theorem

The *Courant-Fischer Theorem* states that the largest eigenvalue is as

$$\lambda_{\max} = \max_{\mathbf{z}} \mathcal{R}_A(\mathbf{z}) = \max_{\mathbf{z}} \frac{\mathbf{z}^T A \mathbf{z}}{\mathbf{z}^T \mathbf{z}}.$$

For the power method, observe that $\beta_m = \mathcal{R}_A(\mathbf{z}^{(m)})$.



Inverse Iteration

Consider if one has an approximate eigenvalue $\lambda \approx \lambda_i$. Inverse iteration modifies the power method to estimate λ_i and its eigenvector $\mathbf{v}^{(i)}$.

Definition

Let $\mathbf{x}^{(0)}$ be an initial guess at eigenvector $\mathbf{v}^{(i)}$. *Inverse iteration* is defined as:

$$\begin{aligned}\mathbf{z}^{(m)} &= \mathbf{x}^{(m)} / \|\mathbf{x}^{(m)}\|_2 \\ \mathbf{x}^{(m+1)} &= (A - \lambda I)^{-1} \mathbf{z}^{(m)} \\ \beta_m &= [\mathbf{z}^{(m)}]^T A \mathbf{z}^{(m)}\end{aligned}$$

Then $\mathbf{z}^{(m)} \rightarrow \mathbf{v}^{(i)}$ and $\beta_m \rightarrow (\lambda_i - \lambda)^{-1}$ as $m \rightarrow \infty$.

(See notes.)



Inverse Iteration

Theorem

Let λ_i and λ_j be the closest and second-closest eigenvalues to λ . Then inverse iteration converges as

$$\|\mathbf{z}^{(m)} - \mathbf{v}^{(i)}\|_{\infty} \leq c \left(\frac{|\lambda_i - \lambda|}{|\lambda_j - \lambda|} \right)^m$$

for some constant $c > 0$, provided that the initial condition is not orthogonal to $\mathbf{v}^{(1)}$.

(See notes for proof.)



Notes on Power Method and Inverse Iteration

- For inverse iteration, one does not necessarily need to compute the inverse matrix $B = (A - \lambda I)^{-1}$. It may be faster to instead solve a linear system for $\mathbf{x}^{(m+1)}$:

$$(A - \lambda I)\mathbf{x}^{(m+1)} = \mathbf{z}^{(m)}$$

- Both the power method and inverse iteration can be implemented efficiently for large sparse matrices using sparse matrix-vector multiplication

Rayleigh Quotient Iteration (RQI)

Can we replace λ for inverse iteration by our estimate β_m ? Yes.

Definition

Rayleigh Quotient Iteration is defined as:

$$\mathbf{z}^{(m)} = \mathbf{x}^{(m)} / \|\mathbf{x}^{(m)}\|_2$$

$$\beta_m = [\mathbf{z}^{(m)}]^T A \mathbf{z}^{(m)}$$

$$\mathbf{x}^{(m+1)} = (A - \beta_m I)^{-1} \mathbf{z}^{(m)}$$

Then $\mathbf{z}^{(m)} \rightarrow \mathbf{v}^{(i)}$ and $\beta_m \rightarrow \lambda_i$ as $m \rightarrow \infty$.

Note: For non-repeated eigenvalues, RQI converges quadratically!

(See code.)



Deflation

How can we find multiple eigenvalues and eigenvectors?

Definition

Let A be a square, symmetric matrix with an eigenvalue λ with right eigenvector $\mathbf{v}$ and left eigenvector $\mathbf{u}$. Then the *deflated* matrix is given by

$$A' = A - \lambda \mathbf{v} \mathbf{u}^T.$$

- ▶ Matrix A' still has a right eigenvector $\mathbf{v}$ and left eigenvector $\mathbf{u}$, but the associated eigenvalue is now: $\lambda - \lambda = 0$.
- ▶ When applied to A' , the power method returns the second-largest eigenvalue of A and its eigenvector. Deflating k times allows us to find the k top eigenvalues and eigenvectors.

(See code for example.)



QR Method to Compute all Eigenvalues

One can simultaneously compute all eigenvalues $\{\lambda_i\}$ of a diagonalizable matrix A via the QR iteration.

Definition

The *QR decomposition* of a matrix A is given by $A = QR$, where R is an upper triangular matrix and Q is a unitary matrix such that $Q^T Q = I$.

Definition

The *QR iteration* for an initial matrix A_0 is defined by

$$Q_m R_m = A_m$$

$$A_{m+1} = R_m Q_m$$

QR Method to Compute all Eigenvalues

Theorem

The sequence of matrices A_m for the QR iteration converges to a diagonal matrix D that contains the eigenvalues on the diagonal. Convergence satisfies the following bound

$$\|D - A_m\| \leq c \max_i \left| \frac{\lambda_{i+1}}{\lambda_i} \right|,$$

where we assume $|\lambda_1| > |\lambda_2| > \dots > |\lambda_n|$.

- ▶ The diagonal entries in A_m converge to the eigenvalues.
- ▶ For symmetric A , the product $Q_1 Q_2 \dots Q_m$ converges to the matrix V of eigenvectors.

(See notes and code.)



Householder Matrices and the QR Decomposition

But how does one construct the $A = QR$ decomposition?

One approach involves Householder matrices.

Definition

Given a vector $\mathbf{w}$ with $\|\mathbf{w}\|_2 = 1$, it's associated *Householder matrix* takes the form: $U = I - 2\mathbf{w}\mathbf{w}^T$

Theorem

U is symmetric and unitary.

(See notes for example and proof.)



Householder Matrices and the QR Decomposition

Householder matrices can be used to convert a column in A into a new column whose lower entries contain zeros.

Example: $A = \begin{bmatrix} 4 & 3 \\ 3 & -3 \end{bmatrix}$

(See notes and code.)